

本地 LLM Agent Prompt/KV 缓存复用的性能评估

稳定前缀比

单纯打开缓存更关键

112.2x

Coding 16K TTFT speedup

37.443s → 0.334s

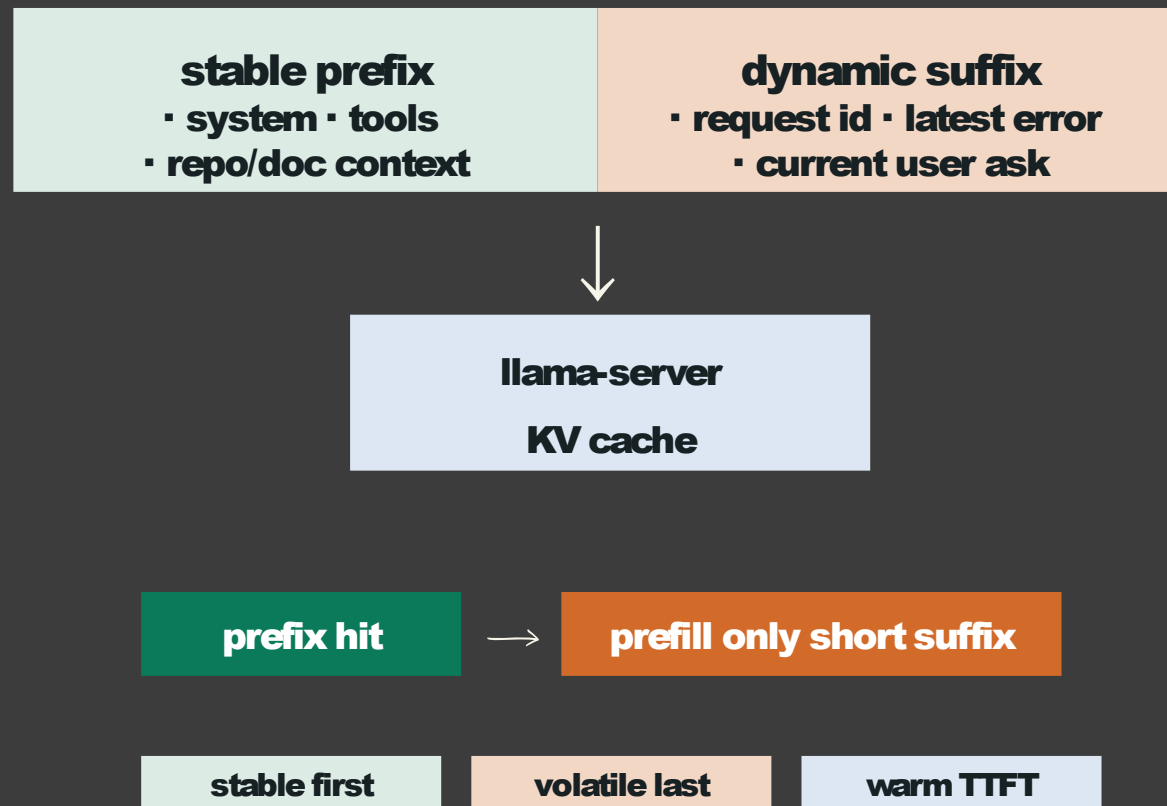
103.7x

RAG 16K TTFT speedup

25.341s → 0.244s

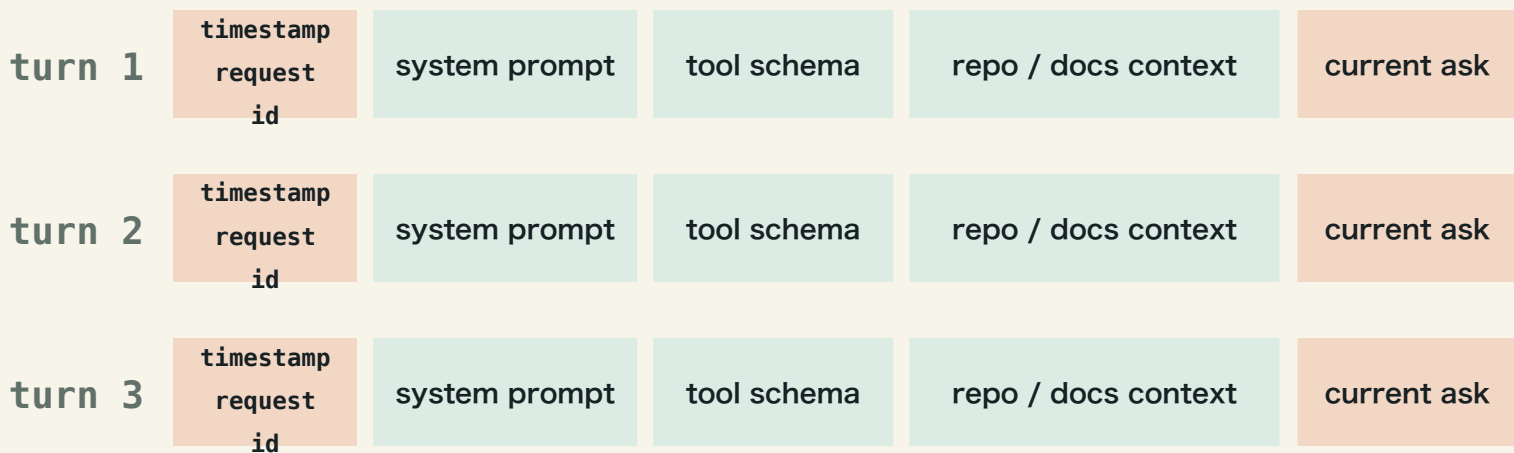
元培学院 胡思成 2400017832

warm turn prompt



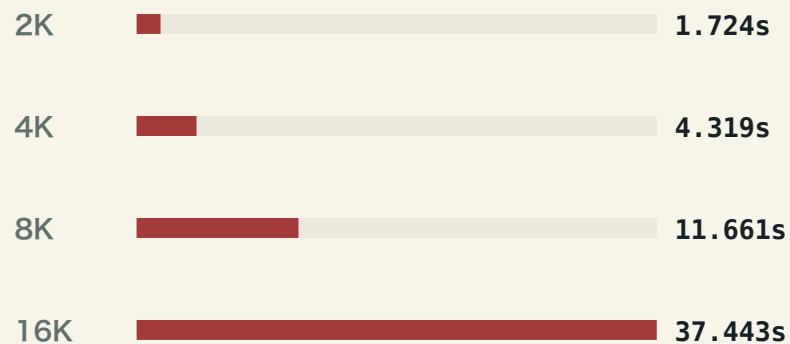
多轮 Agent 反复提交长上下文, warm turn 仍会被重复 prefill 拖慢

三轮请求里, 稳定上下文占主体; 变化通常只是一小段元数据和当前任务



如果开头字段每轮变化,
后面的大段稳定内容也**无法**成为同一条命中中的公共前缀

S0 warm TTFT ramp (Coding)



首 **token** 前的等待随 **prompt** 长度急剧上升,
在 **16K Coding** 中, **TTFT** 已经占总延迟的绝大部分

缓存收益来自可复用前缀，开关本身不等于命中

$$T_{\text{TTFT}} \approx T_{\text{prefill}}(L_{\text{new}}) + T_{\text{decode},1} + T_{\text{overhead}}$$

prefill(L_{new}) + decode step 1 + fixed overhead

$$L_{\text{new}} = L_{\text{prompt}} - L_{\text{reuse}}$$

长上下文差异主要来自 **prefill**
能复用多少前缀，就少算多少 token

S1: volatile field first

dynamic

stable context
cannot be matched as prefix

$$L_{\text{reuse}} \approx 0$$

S2: stable prefix first

reused stable prefix

new suffix

L_{new} stays small

系统指令、工具定义、仓库摘要、固定文档应排在前面，并跨轮保持字节级稳定

分别验证缓存开关、prompt 布局 和 reuse 参数的影响

S0-S3 strategy matrix

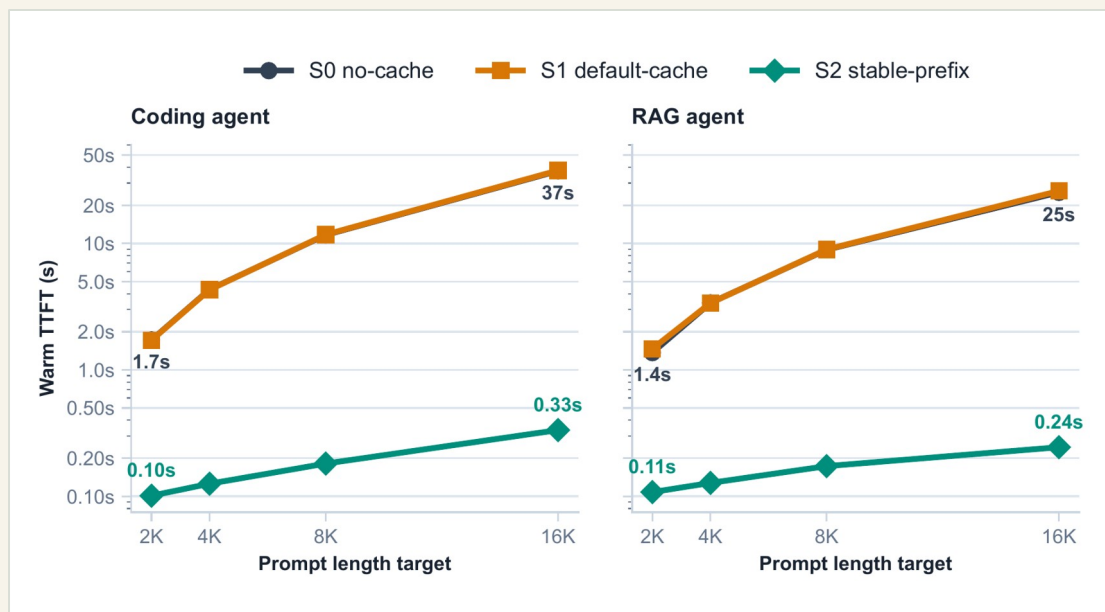
S0	no_cache	关闭缓存；完整上下文每轮重新 prefill
S1	default_cache	动态字段前置；缓存开启但公共前缀被破坏
S2	stable_prefix	稳定内容前置、动态字段后置；主实验目标策略
S3	cache-reuse sweep	沿用 S2 布局，扫描 N=0 / 64 / 128 / 256 / 512

fixed environment

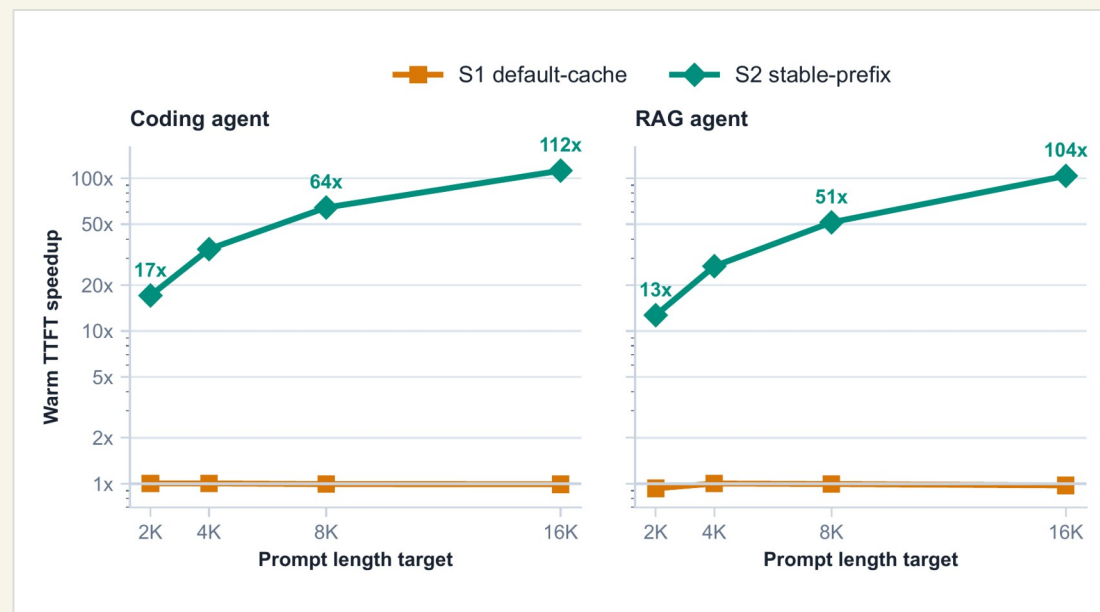
硬件	Apple M5 32 GB unified memory
模型	Qwen2.5-Coder-3B-Instruct Q4_K_M
后端	llama-server, Metal OpenAI-compatible API
请求	temperature=0, max_tokens=64, --parallel 1
负载	Coding agent + RAG agent
长度	2K · 4K · 8K · 16K

20 requests
report warm turns 2-20 median

稳定前缀把长上下文 warm TTFT 从几十秒压到百毫秒级



Warm TTFT by length



TTFT speedup by length

16K 下, Coding: 37.443s → 0.334s ; RAG: 25.341s → 0.244s

上下文越长, 重复 prefill 越贵, S2 的相对收益越大

TTFT 降低后，瓶颈转向 decode

112.2x

16K Coding TTFT speedup

S0 37.443s

S2 0.334s

S2 p95 0.359s

103.7x

16K RAG TTFT speedup

S0 25.341s

S2 0.244s

S2 p95 0.265s

S2 之后的总延迟组成

Coding

TTFT
0.334s

decode ≈ 1.934s

2.268s total

RAG

TTFT
0.244s

decode
≈ 0.386s

0.630s total

What changed?

S0/S1

TTFT 与总 prompt 长度同步上升，
16K 已经进入几十秒量级

S2

warm turn 只处理末尾短后缀，
TTFT 与总长度弱相关

p95

S2 p95 接近中位数，
说明第 2-20 轮持续命中，而非少数偶然值

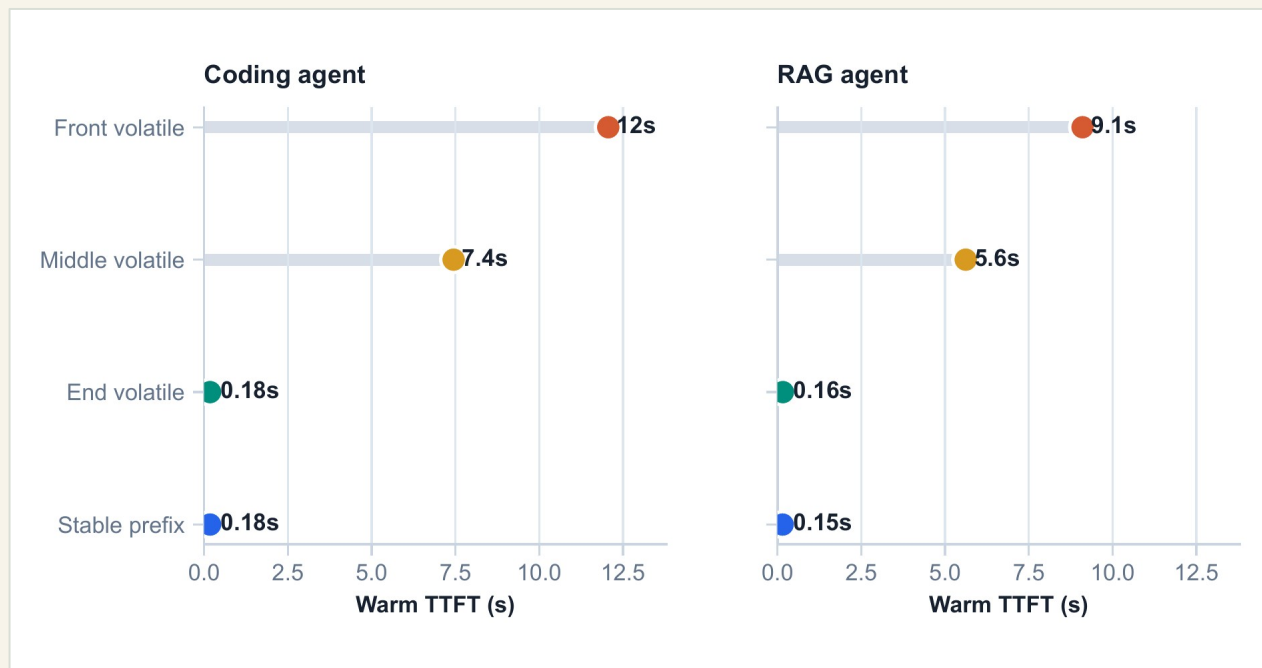
decode

prefill 被跳过后，
长回答场景会更早暴露逐 token 生成吞吐

prompt cache 的核心价值是让请求更快进入正常生成阶段

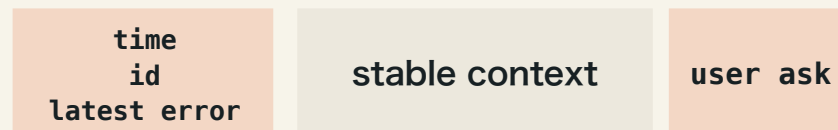
端到端收益还会受输出长度影响

动态字段可以存在，但不能打断可复用前缀



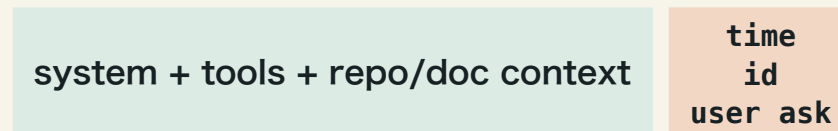
8K prompt 下, front_volatile 把 TTFT 拉回 9-12 秒量级 end/stable_prefix 保持 0.15-0.18 秒

bad layout



prefix mismatch at byte/token 1

good layout

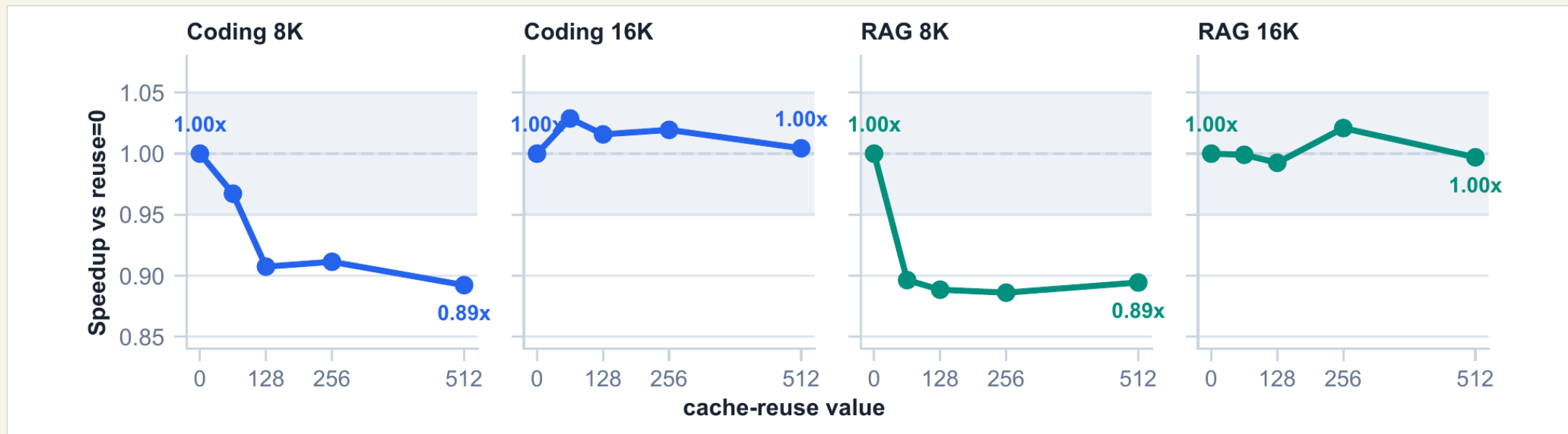


long exact prefix remains reusable

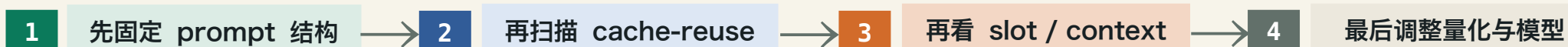
移动动态字段不改变任务信息量

却能把 warm TTFT 从秒级降到百毫秒级

--cache-reuse 仅作为边际调参，不是结构性解法



在稳定前缀已建立时，8K 的非零 reuse 普遍慢约 10%-11%；16K 最佳点只快约 2%-3%



缓存状态需换成命中收益， 否则只是在占内存



2.28-3.36 GiB

S2 peak RSS range

在 **2K-16K** 范围内，资源开销可控，
同时带来百倍级 **TTFT** 加速

S1 内存占用更高，但 TTFT 没有改善，
因为前缀无法稳定命中

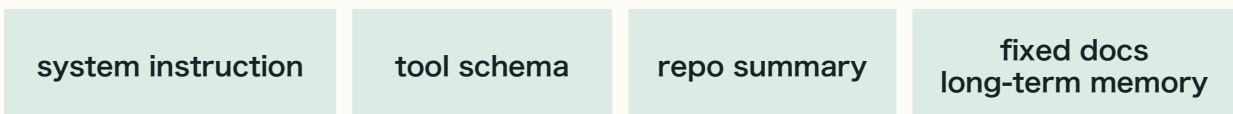
S2 同样使用缓存，
但稳定前缀把缓存状态转化为 *warm-turn* 性能

RSS 包含模型权重、Metal 缓冲区、运行时分配和缓存管理状态，
不能直接等同于逐 token KV 理论大小

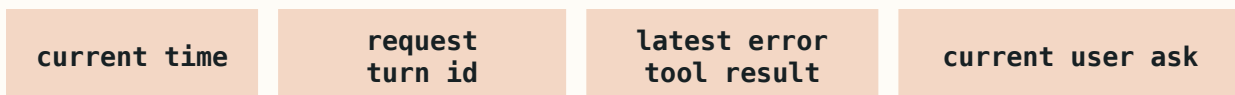
关键不是“占用更多缓存”，
而是缓存命中是否稳定

把 prompt 结构视作性能接口

recommended prompt template



stable prefix: byte-level stable across turns



dynamic suffix: allowed to change

implementation checklist

- 1 稳定内容前置**：系统指令、工具定义、仓库摘要、固定文档、长期记忆
- 2 动态内容后置**：request id、turn id、最新错误、工具返回、用户请求
- 3 保持跨轮稳定**：避免随机顺序 JSON、临时路径、计数器混入前缀
- 4 优化顺序**：稳定前缀协议优先，再调 reuse、slot、上下文长度和量化

先让 cache 有东西可命中，再进一步实验 cache 参数怎么调



谢谢

谢谢老师和同学们的倾听，欢迎指正。